Programación con Objetos 1

Conceptos fundamentales de la programación con objetos

Autores:

- Facundo Javier Gelatti — javiergelatti@gmail.com
- Máximo Prieto — maximo.prieto@gmail.com

Este documento está bajo una licencia CC BY 4.0

Última modificación: 2026-03-29

Índice

Los paradigmas de programación **2**
Una primera aproximación al concepto 2
El paradigma imperativo, procedural-estructurado 2
El paradigma orientado a objetos: una forma diferente de pensar 6
Definiendo al paradigma de objetos desde sus postulados elementales **7**
¿Qué es un programa? 7
¿Qué es un objeto? 8
¿Qué es un mensaje? 12
¿Qué es una colaboración? 13
¿Qué es un colaborador? 14
¿En qué consiste la estructura de un objeto? 15
¿Qué es un protocolo? 18
¿Qué es un método? 20
Resumen **22**
Programa orientado a objetos 22
Objeto 22
Colaboración 23
Colaborador 23
Mensaje 23
Protocolo e implementación 23
Método 24
Esquema conceptual 24
El paradigma como forma de pensar 24

Los paradigmas de programación

Ahora que ya dimos los primeros pasos con objetos, vamos a tomarnos un momento para reflexionar más en profundidad sobre qué significa estar trabajando dentro del *paradigma orientado a objetos* y cómo se diferencia de otras formas de programar que ya conocés.

Una primera aproximación al concepto

Un paradigma de programación es una forma particular de entender qué es un programa y cómo debe construirse. Esto va más allá de un lenguaje específico o una herramienta particular: un paradigma representa un conjunto de ideas y conceptos fundamentales que guían la manera en la que pensamos sobre los problemas computacionales y sus soluciones.

Hasta ahora, en materias anteriores, has estado programado usando el *paradigma imperativo*, específicamente en su variante *procedural-estructurada*, con Gobstones. Repasemos en qué consiste y cómo se diferencia del paradigma orientado a objetos.

El paradigma imperativo, procedural-estructurado

En un enfoque *imperativo*, un programa se concibe como una *secuencia de instrucciones* que modifican el estado de la computadora y/o producen una salida. Es como darle órdenes paso a paso: "hacé esto, después aquello, luego lo otro" (de ahí viene su nombre, como el *modo verbal imperativo*). La idea central es que estamos indicándole a la computadora *cómo* hacer algo, detallando cada paso del procedimiento.

Por ejemplo, un programa imperativo simple en Gobstones podría verse así:

```
program {
  VaciarTablero()
  IrAlBorde(Sur)
  IrAlBorde(Oeste)
  DibujarCuadradoNegroDeLado3()
}
```

Aquí le estamos diciendo a la computadora exactamente qué hacer: primero vaciar el tablero, luego ir al borde sur, después al borde oeste y finalmente dibujar un cuadrado negro. Cada línea es una instrucción que se ejecuta en orden.

❦

En sus formas más primitivas, un programa imperativo puede tener saltos arbitrarios en el flujo de ejecución mediante el uso de instrucciones como `goto`[1], lo que puede generar código difícil de entender.

Imaginá cómo podría verse un programa en Gobstones si tuviera este tipo de instrucciones:

```
program {
  Poner(Rojo)
  goto salto

  proceso:
  if (not hayBolitas(Rojo)) { goto fin_proceso }
  Sacar(Rojo)
  goto proceso
  fin_proceso:
  Poner(Verde)
  goto fin

  salto:
  Poner(Rojo)
  goto proceso

  fin:
  Poner(Azul)
}
```

En este ejemplo ficticio, vemos cómo el flujo del programa salta de una parte a otra usando `goto`, haciendo que sea difícil seguir la lógica del programa.

💭 ¿Qué hace el programa? ¿Cómo lo refactorizarías para que no use `goto`?

Afortunadamente, nuestra manera de programar evolucionó hacia formas más ordenadas.

La programación estructurada

En su famoso artículo "Go To Statement Considered Harmful" (1968), Dijkstra argumentó que el uso descontrolado del `goto` dificulta nuestro razonamiento con respecto al comportamiento de los programas. Según él, el uso de la instrucción `goto` es nocivo porque impide “mantener un sistema claro de coordenadas mentales para describir el progreso de un programa”; es decir, la secuencia del texto del programa deja de corresponderse con la secuencia de ejecución.

[1] Viene del inglés “*go to*”: ir a. La instrucción recibe una etiqueta que corresponde al lugar del programa desde donde se debe continuar la ejecución.

De allí surgió la *programación estructurada*, como una propuesta dentro del paradigma imperativo, que plantea que todo programa debe construirse combinando únicamente tres estructuras básicas:

1. **Secuencia**: ejecutar instrucciones una después de otra.
2. **Selección**: elegir entre diferentes caminos (ej. usando `if`/`else`).
3. **Repetición**: ejecutar un bloque de código varias veces (ej. usando `while` o `repeat`).

Este enfoque busca hacer que los programas sean más claros y comprensibles, eliminando instrucciones como el `goto`. Partiendo del ejemplo anterior, que usaba `goto`, podemos transformarlo —manteniendo su comportamiento— para que use estructuras de control como `while`, y desenvolviendo los saltos para que queden como una secuencia:

```
program {
  Poner(Rojo)
  Poner(Rojo)

  while (hayBolitas(Rojo)) {
    Sacar(Rojo)
  }

  Poner(Verde)
  Poner(Azul)
}
```

Dejamos a tu criterio comparar ambos programas y decidir cuál de los dos es más entendible y claro.

La programación procedural

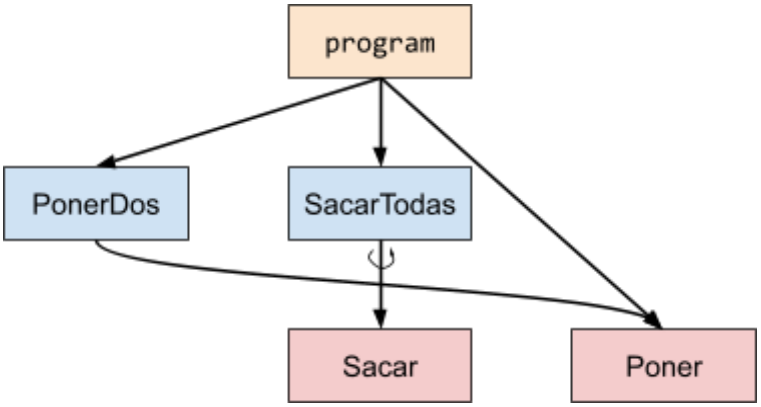
La programación *procedural* agrega otro nivel de organización: divide el programa en rutinas o procedimientos que pueden llamarse entre sí. Esto permite agrupar instrucciones relacionadas y reutilizar código. Tomando el ejemplo que venimos desarrollando, podríamos descomponerlo en procedimientos de la siguiente manera:

```
program {
  PonerDos(Rojo)
  SacarTodas(Rojo)
  Poner(Verde)
  Poner(Azul)
}
```

```
procedure PonerDos(color) {
  Poner(color); Poner(color)
}

procedure SacarTodas(color) {
  while (hayBolitas(color)) {
    Sacar(color)
  }
}
```

En este ejemplo, el programa principal (`program`) *llama* a varios procedimientos. Esto termina formando una estructura jerárquica:



Este enfoque, en su momento, fue revolucionario: el enfoque imperativo, en su versión procedural-estructurada, ha influido profundamente (para bien o para mal) en cómo pensamos sobre la programación. Su impacto es tan significativo que ha afectado la definición hegemónica de lo que consideramos un "programa" *en general*.

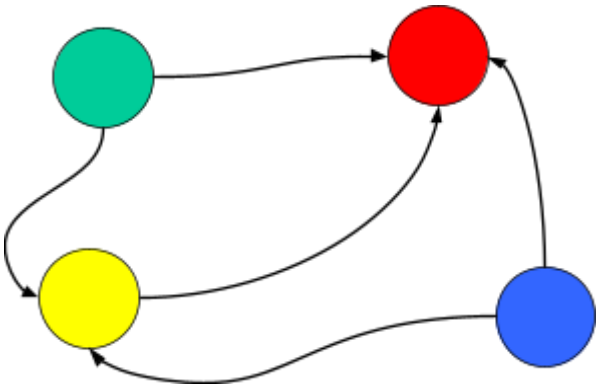
Esta influencia tiene mucho que ver con que la arquitectura de las computadoras actuales, basada en el modelo de Von Neumann, está diseñada para ejecutar *secuencias de instrucciones*[2]. Esto ha reforzado la idea —errónea— de que programar *siempre* consiste necesariamente en *especificar una secuencia de pasos que la computadora debe seguir*. Por eso, una de las definiciones más conocidas de “programa” es "una secuencia de instrucciones que le dice a una computadora qué es lo que tiene que hacer". Esta definición, aunque válida en muchos contextos, está arraigada en el enfoque imperativo y refleja su predominancia en nuestra forma de pensar a la programación.

[2] Sólo hay que observar el ciclo *fetch-decode-execute* que sigue el procesador: buscar una instrucción, decodificarla, ejecutarla, y repetir…

El paradigma orientado a objetos: una forma diferente de pensar

El paradigma orientado a objetos propone una visión distinta: en lugar de pensar en términos de instrucciones y procedimientos, pensamos en términos de *objetos que colaboran entre sí*.

Un programa, en lugar de ser una secuencia de órdenes, es una comunidad de objetos que se comunican enviándose mensajes:



Del "cómo" al "qué"

Una diferencia clave entre estos paradigmas es que en la programación imperativa nos enfocamos en el "cómo": cómo lograr algo paso a paso, detallando cada instrucción que debe ejecutarse. En cambio, en la programación orientada a objetos (y en otros paradigmas *declarativos*) nos enfocamos en el "qué": qué queremos hacer y a qué objeto podemos pedirle que lo haga por nosotros. Luego, es tarea de cada objeto determinar “cómo” hace lo que le pedimos que haga: la implementación específica de cómo cada objeto cumple su responsabilidad queda *encapsulada* dentro del propio objeto, invisible para los demás.

Entonces… ¿qué es realmente un programa?

Llegado a este punto, debería hacernos ruido que tengamos más de una definición de lo que es un "programa". Es decir, si tenemos en cuenta que el concepto es uno solo, esperaríamos tener una única definición y no varias. Entonces, ¿cuál es la correcta? ¿Un programa es una secuencia de instrucciones, o es una comunidad de objetos que colaboran entre sí?

La verdad es que ambas descripciones son válidas dentro de sus respectivos paradigmas. Pero, ¿sería posible tener una definición superadora —más abarcativa y abstracta— de “programa”? Vamos a dejar esta discusión abierta, pero queda como invitación a reflexionar sobre qué significa realmente programar. Por ahora, vamos a seguir explorando las características del paradigma orientado a objetos.

Definiendo al paradigma de objetos desde sus postulados elementales

Luego de esa breve introducción, vamos a construir ahora una definición más formal del paradigma de objetos. En lugar de presentar directamente todos los conceptos, vamos a derivarlos a partir de un axioma fundamental: la definición de programa orientado a objetos. Esta aproximación nos va a permitir entender en qué consiste es cada elemento del paradigma, y también por qué existe y qué papel juega en el conjunto.

¿Qué es un programa?

Para empezar, retomemos la definición de programa:

> Un **programa orientado a objetos** es un c*onjunto de objetos que colaboran enviándose mensajes*.

Esta definición, aunque breve, tiene implicaciones profundas:

1. **Sólo hay objetos**: no existen elementos computacionales de otra naturaleza.
2. **Lo único que pueden hacer los objetos es enviar y recibir mensajes**: toda interacción ocurre a través de mensajes.

Por lo tanto, si queremos que algo suceda en el programa, necesitamos (1) un objeto responsable de hacer esa tarea, y (2) otro objeto que le envíe un mensaje para pedirle que la realice.

Notemos que esta definición es *pura* y *minimal*. Es *pura* porque es autónoma, es decir, no menciona ningún concepto por fuera del paradigma: no existen los “datos”, los “procedimientos”, las “funciones”, las “estructuras de control”, el “if”, etc. Y es *minimal* porque evita introducir elementos no esenciales —en el sentido de superfluos o inferibles— del paradigma[3].

A partir de esta definición básica, vamos a construir el resto de los conceptos del paradigma, comprendiendo de dónde surge cada uno y qué papel cumple en el conjunto.

Para avanzar, necesitamos profundizar en los conceptos que aparecen en la definición de programa:

- ¿Qué es un **objeto**?
- ¿Qué es un **mensaje**?
- ¿Qué significa **colaborar**?

Comencemos por definir qué es un objeto.

[3] En particular, prescinde de muchas cosas que suelen mencionarse en otras introducciones a la programación orientada a objetos, como “clases”, “herencia”, “encapsulamiento”, etc. A estas nociones las iremos derivando naturalmente a medida que avancemos en nuestra exploración.

¿Qué es un objeto?

Un objeto se puede entender desde dos perspectivas complementarias. Desde una perspectiva técnica, basados en la definición de programa, los objetos van a ser los elementos computacionales básicos que utilizamos para construir programas: son como las "células" con las que componemos nuestras soluciones de software[4].

Sin embargo, nosotros preferimos dar una definición conceptual, que conecta nuestro programa con la realidad que pretende representar:

> Un **objeto** es una *representación esencial de una cosa de un dominio de interés*.

De esta definición surgen nuevos conceptos que necesitamos aclarar:

- ¿Qué entendemos por **cosa**?
- ¿A qué nos referimos con **dominio de interés**?
- ¿Qué significa **representación esencial**?

¿Qué entendemos por cosa?

Resulta curioso que, como parte de esta explicación, necesitemos definir qué es "cosa". Si buscamos definiciones de “objeto” en la Web, muchas mencionan la palabra “cosa” dando por sentado que sabemos perfectamente qué es una cosa. Pero aquí no queremos dar nada por sobreentendido, especialmente cuando “objeto” y “cosa” son sinónimos en el habla cotidiana. Así que vamos a ser explícitos incluso con este concepto tan aparentemente obvio.

> 💭 Hablando de obviedad, ¿cómo definirías “cosa”, sin usar ningún sinónimo o palabra no definida como “objeto”, “entidad” o “un algo”?
> A primera vista parece sencillo, pero rápidamente nos damos cuenta de que no es tan trivial[5].

Para definir qué es una "cosa", nosotros vamos a utilizar un enfoque *lingüístico*, fundamentando la existencia de las cosas en nuestra capacidad para referirnos a ellas mediante el lenguaje. Si bien esta es sólo una de las posibles maneras de entender este concepto, resulta muy práctica para nuestros fines como programadores:

> Una **cosa** es *todo aquello acerca de lo cual podemos decir algo*.[6]

---

[4] Esta metáfora resulta especialmente relevante: las células se comunican entre sí mediante mensajes químicos, de forma similar a los objetos. Además, así como las células se especializan (en nerviosas, sanguíneas, epiteliales) y se agrupan formando tejidos, órganos y sistemas más complejos, nuestros objetos se especializan para cumplir diferentes roles y colaboran formando subsistemas, integrándose de manera armónica para constituir programas completos.

[5] Esto recuerda a la frase de San Agustín de Hipona: "¿Qué es, pues, el tiempo? Si nadie me lo pregunta, lo sé; si quiero explicarlo a quien me lo pide, no lo sé".

[6] El filósofo y matemático Gottlob Frege daba una definición alternativa para “cosa”: *todo lo que es susceptible de ser designado por un nombre propio* (Frege, 2017). ¿Serán equivalentes ambas definiciones? Dejamos la demostración o refutación como ejercicio para pensar.

Entonces, pensemos un poco. Según esta definición, ¿cuáles de estas son *cosas*?

- Una mesa
- El número uno
- Un unicornio
- El amor
- La justicia
- Un depósito bancario
- Un error de compilación

Bueno… todas ellas son cosas, porque podemos hablar sobre cada una, por ejemplo para describirlas o referirnos a ellas. “El número uno es el primer número natural”. “Un unicornio, aunque no exista físicamente, es un caballo con un cuerno”. “El amor es un sentimiento profundo”. “Un depósito bancario es una operación que incrementa el saldo de una cuenta”[7].

Esta definición es importante porque cuando programamos a menudo representamos y trabajamos con elementos que no tienen existencia física concreta: un préstamo, una reserva de hotel, un proyecto... Todos ellos son cosas sobre las que podemos hablar y que tienen un propósito en su contexto, aunque no sean tangibles.

¿A qué nos referimos con dominio de interés?

Ya que tenemos claro qué es una cosa, ahora podemos decir que:

> Un **dominio** es un *conjunto de cosas.*

Cuando hablamos de “dominio de interés”, nos referimos específicamente al conjunto de cosas en el que nos estamos enfocando, que “nos interesa”, sobre las cuales estamos trabajando con nuestro programa. También podríamos decir “dominio del problema”, que pone el foco en que es el conjunto de cosas que son relevantes para el problema que estamos resolviendo con nuestro programa; o “dominio de negocio”, que pone el foco en que estamos trabajando sobre un negocio.

Una característica de nuestra profesión es que, como programadores, generalmente trabajamos sobre dominios preexistentes, creados por la sociedad para algún fin específico. Es decir, a diferencia de los científicos, que investigan dominios naturales o sociales para descubrir sus leyes y propiedades, nosotros construimos programas para dominios ya conocidos y que tienen propósitos específicos.

Estos dominios suelen ser dominios de negocio: bancos, hospitales, compañías de seguros, librerías, bibliotecas, etc. Al ser creaciones humanas, cada cosa en estos dominios tiene un propósito definido, una razón de existir. Esto es fundamental para nuestro trabajo, porque significa que nuestros programas siempre son susceptibles de capturar ese propósito, esa utilidad para la cual fueron creadas esas cosas.

[7] A pesar de que todas estas afirmaciones son verdaderas, si prestamos atención a la definición de “cosa”, veremos que no es relevante si lo que se dice es verdadero o falso: lo que cuenta es que se lo pueda decir.

¿Qué significa representación esencial?

Cuando hablamos de "representación esencial", nos referimos a capturar la esencia de una cosa, no todos sus detalles. Distinguimos entre una *representación esencial* (de una cosa) y una *representación arbitraria*. Si dibujara una cruz y dijera "esto representa mi auto":

…estaría creando una representación completamente arbitraria. Podemos aceptarla y saber que la cruz representa el auto, pero no hay nada en ella que capture una idea de lo que es ese auto. Si otra persona viera ese dibujo sin esta explicación, no tendría forma de saber qué representa.

En cambio, una representación esencial captura justamente aquello que hace que la cosa sea lo que es. En lugar de depender de convenciones o de explicaciones adicionales, refleja la naturaleza intrínseca de lo que está representando. Con esto vamos con otra definición:

> La **esencia** de una cosa es *lo que hace que esa cosa sea lo que es, y no otra cosa diferente*. Es lo que permanece invariante a lo largo del tiempo, más allá de los cambios accidentales.

Para entender mejor este concepto, consideremos algunos ejemplos:

- Tener exactamente cuatro lados iguales y cuatro ángulos rectos forma parte de la esencia de un cuadrado. Si una figura no tiene estas propiedades, simplemente no es un cuadrado.
- Por el contrario, la ropa que estás usando hoy es una propiedad *accidental* tuya: podés cambiarte y vas a seguir siendo la misma persona.

Al margen: sobre el concepto de “esencia”

La idea de "esencia" tiene una larga historia en la filosofía, remontándose a Platón y su *teoría de las Formas*. Para Platón, la esencia de algo corresponde a su *Forma ideal*, que existe en un plano abstracto y del cual las cosas particulares en el mundo físico son solo copias o instancias imperfectas. Aristóteles, su discípulo, reformuló estas ideas proponiendo una distinción más concreta entre las *propiedades esenciales* y *accidentales* de una cosa.

Estos conceptos fueron centrales en la filosofía medieval y continúan siendo influyentes en el pensamiento contemporáneo, aunque con diversas reinterpretaciones y matices[8]. Si bien la filosofía moderna ha cuestionado la noción de esencias fijas e inmutables, especialmente cuando se llevan a extremos *esencialistas*, el concepto sigue siendo valioso como herramienta analítica en diversos campos, incluida la programación.

[8] Para poner un ejemplo de una observación posible, W. V. Quine decía que en realidad las cosas no tienen propiedades esenciales o accidentales en sí mismas, sino que depende del interés (contextual) que tengamos sobre cada propiedad (Quine, 1980).

En la programación orientada a objetos vamos a tomar un enfoque particular para entender qué es para nosotros la esencia de una cosa, basado en el *pragmatismo*. El pragmatismo es una corriente filosófica que surgió a finales del siglo XIX en Estados Unidos. Esta filosofía propone que el significado y la verdad de las ideas deben evaluarse según sus consecuencias prácticas y su utilidad. Para los pragmatistas, el valor de un concepto no reside en su correspondencia abstracta con la realidad, sino en cómo funciona en la práctica y qué tan útil resulta para lograr objetivos concretos.

Siguiendo esta línea de pensamiento, en nuestro contexto diremos que una cosa *es* en tanto y en cuanto *nos sirve para algo* (o sea, una cosa *es* sí y sólo sí *podemos hacer algo con ella*). Es decir, la esencia de una cosa es su utilidad, para qué está, para qué sirve.

> La **esencia** de una cosa es *lo que hace que esa cosa sea lo que es, y no otra cosa diferente*, y está expresada a través de su *utilidad*: lo que podemos hacer con ella, para qué sirve, qué rol cumple en su contexto.

Entonces, siguiendo estas ideas, si nos preguntamos por ejemplo:

- ¿Qué hace que un lápiz sea un lápiz? Será su capacidad para escribir o dibujar sobre papel.
- ¿Qué hace que una caja de ahorros sea una caja de ahorros? Será su capacidad para almacenar dinero, permitir depósitos y extracciones, mantener un saldo, etc.

En cada caso, lo esencial se relaciona con el propósito para el cual la cosa fue creada, su utilidad en el contexto en que existe. Este enfoque pragmático tiene una relación directa con el concepto de objeto: si algo es un objeto, *entonces entiende algún mensaje, y por lo tanto le podemos pedir que haga algo por nosotros*. Si algo no entiende ningún mensaje, entonces no puedo hacer nada con eso, y luego *no es* un objeto (al menos no en el sentido que le estamos dando nosotros a la palabra “objeto”).

Al margen: sobre el pragmatismo

La visión pragmatista es controvertida en muchos contextos filosóficos, en el sentido de que parece limitarnos sobre cómo pensar en la realidad, evitando que consideremos aspectos “no útiles” que también forman parte de ella.

Entrando más en detalle, el pragmatismo puede ser polémico por varias razones. Por una parte, reduce la "realidad" de las cosas a su *utilidad instrumental*, lo que limita nuestra capacidad de pensar sobre cosas que no tienen una funcionalidad clara o que no están diseñadas para hacer algo específico. Por otra parte, podría interpretarse como una visión excesivamente *antropocéntrica*, donde las cosas solo "son" en relación a lo que *los humanos* podemos (o se nos ocurre) hacer con ellas.

Sin embargo, dado que estaremos desarrollando programas referidos a, como ya hemos mencionado, dominios creados por la sociedad (es decir, *por humanos*) *con propósitos específicos*, las cosas en estos contextos siempre existen para cumplir alguna función, para ser utilizadas de alguna manera. Por lo tanto, las restricciones que nos impone esta

visión *en general* no parecieran limitarnos tanto *en este caso*, por lo menos en el sentido de lo que nos importa como programadores[9].

En resumen, *representar arbitrariamente* a una cosa es simplemente definir un signo para ella. Por otro lado, *representar esencialmente* a una cosa es identificar lo esencial (en nuestro caso, qué puedo hacer con esa cosa) e incluirlo como parte inseparable de su representación. Esto debería revelar lo que esa cosa *es*, y *su propósito o utilidad*. Además, en el caso de los dominios de negocio, esto nos dice *por qué existe esa cosa*.

📝 Nota: a veces vamos a usar la palabra “modelar” como sinónimo de “representar”.

¿Qué es un mensaje?

Como ya vimos, los mensajes son el mecanismo a través del cual los objetos se comunican entre sí dentro del paradigma orientado a objetos. Cuando decimos que "le enviamos un mensaje a un objeto", estamos pidiéndole que realice alguna acción o que nos provea alguna información. El mensaje especifica *qué* queremos que el objeto haga, aunque no *cómo* debe hacerlo; esto último queda encapsulado dentro del objeto.

Podemos definir “mensaje” de la siguiente manera:

> Un **mensaje** es la manera en que un objeto le solicita a otro que haga algo.

Recordemos, además, que los mensajes son la única forma de interacción posible entre los objetos. Sin mensajes, los objetos no pueden comunicarse ni hacer nada útil.

Los mensajes se especifican a través de su nombre (o *selector*) y de los *colaboradores externos*, que son objetos que enviamos junto con el mensaje para completar la petición que estamos haciendo (es decir, nos permiten terminar de especificar concretamente qué queremos que haga el objeto receptor).

Son ejemplos de mensajes:

- `size`
- `+ 4`
- `beginsWith: $h`
- `between: 1 and: 10`

Y los nombres o *selectores* de estos mensajes son, respectivamente:

- `#size`
- `#+`
- `#beginsWith:`
- `#between:and:`

[9] Para pensar: ¿se podría cuestionar esto? ¿Será cierto en general, o será que se cumple en las aplicaciones de negocios que solemos construir, pero no en otro tipo de dominios? ¿Qué ejemplos o contraejemplos se nos ocurren?

Al margen: usos cotidianos de la palabra “mensaje”

Habiendo entendido la definición de *mensaje*, hay que aclarar que a veces usamos la palabra “mal y pronto”, mencionando sólo al *selector* del mensaje (es decir, sin los colaboradores externos). Si esto sucede, podemos estar cayendo en dos casos:

A. Que sea un atajo para no tener que mencionar a los colaboradores externos.
   Por ejemplo, si estamos considerando el siguiente código:
   `1 + 2`
   y decimos “ahí se envía el mensaje `#+`”, nos estamos refiriendo —de forma abreviada— al mensaje `+ 2`.

B. Que nos estemos refiriendo a un *conjunto de mensajes*: todos los mensajes posibles, que tengan sentido, cuyo nombre coincida con el selector mencionado.
   Por ejemplo, si decimos “todos los números entienden el mensaje `#+` para sumarse con otro número”, nos estamos refiriendo a *cualquier* mensaje de la forma `+ n`, donde `n` es un número.

¿Qué es una colaboración?

Si el mensaje es una solicitud, por sí sola no representa una colaboración. La colaboración se da cuando un objeto efectivamente le envía un mensaje a otro. Por lo tanto:

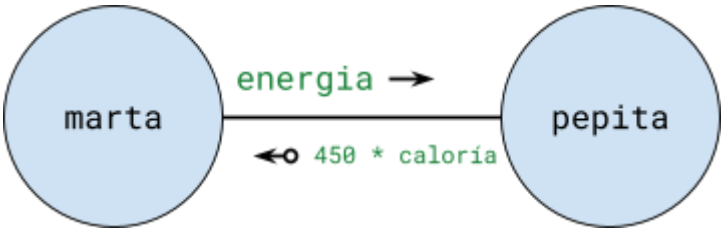
> Una **colaboración** es la interacción concreta de un objeto con otro mediante el envío de un mensaje. Para que un objeto pueda colaborar con otro, debe *conocerlo*.

Es decir, la colaboración es la expresión de la comunicación entre objetos. Si los mensajes son el "lenguaje" con el que se comunican los objetos, las colaboraciones son las "conversaciones" concretas que ocurren entre ellos.

En toda colaboración existen los siguientes elementos:

1. Un **emisor** del mensaje (el objeto que inicia la comunicación[10]).
2. Un **receptor** del mensaje (el objeto que recibe el mensaje y debe responder[11]).
3. El **mensaje** enviado.
4. Una **respuesta** (que siempre es otro objeto).

Por ejemplo, en el siguiente caso:



[10] Este objeto siempre será `self` en el contexto en donde se *envía* el mensaje (¿por qué?).
[11] Este objeto siempre será `self` en el contexto en donde se *recibe* el mensaje (¿por qué?).

Hay una colaboración, donde el emisor es el objeto `marta`, el receptor es `pepita`, el mensaje es `#energía`, y la respuesta es `450 * caloría`.

💡 Una curiosidad: puede suceder que un mismo mismo objeto sea emisor, receptor, colaborador externo y respuesta en una colaboración. Por ejemplo, cuando queremos calcular el cuadrado del número uno, y hacemos `1 squared`, termina pasando lo siguiente:

```
squared
    ↑ self * self
```

En ese caso, `self` es el número `1`. Por lo tanto, `1` se enviará el mensaje `* 1` a sí mismo, y terminará respondiendo `1`.

❦

Las colaboraciones tienen las siguientes características que definen cómo funcionan:

- Son *dirigidas*: cada mensaje va dirigido a un objeto específico, no se envía a múltiples objetos simultáneamente.
- Son *sincrónicas*: el objeto emisor se queda esperando hasta recibir la respuesta.
- *El receptor desconoce al emisor*: la reacción del receptor será siempre la misma sin importar quién le envía el mensaje.

Además, en Smalltalk *siempre hay una respuesta*, incluso cuando no la utilizamos. Si no especificamos explícitamente una respuesta con `↑`, se responderá con `self`.

Una cosa que quedó pendiente: en la definición de colaboración mencionamos que un objeto sólo puede colaborar con otro si es que lo conoce. Dicho de otra forma, no puedo enviarle un mensaje a un objeto desconocido. Tenemos que definir, entonces, cómo se conocen los objetos entre sí…

¿Qué es un colaborador?

Desde la perspectiva de un objeto particular, le llamamos “colaborador” a cualquier otro objeto con quien ese objeto pueda colaborar. Es decir, son los objetos a los cuales conoce, y a quienes les puede enviar mensajes. Cuando un objeto le envía un mensaje a un colaborador, supone que éste cumplirá un determinado *rol* (es decir, que podrá responder a ciertos mensajes; entre ellos el mensaje enviado). Luego:

> En el contexto de un objeto, un **colaborador** es *un objeto conocido, que cumple un determinado rol*.

¿Cómo conoce un objeto a otros?

La relación de conocimiento es la única forma en que los objetos pueden vincularse entre sí dentro del paradigma de objetos. Técnicamente, podemos definirla como:

Una **relación de conocimiento** es un vínculo que, en un contexto determinado, *asocia un nombre con la identidad de un objeto específico*, permitiendo que nos refiramos a él usando ese nombre. Cuando un objeto conoce a otro, decimos que tiene una *referencia* a ese objeto específico, y puede enviarle mensajes a través del nombre asignado.

Hay tres maneras principales en que un objeto puede establecer una relación de conocimiento con otro:

1. **Como colaborador interno**: son objetos que forman parte del "staff" permanente del objeto. Están siempre disponibles mientras el objeto exista y son referenciados mediante nombres que pertenecen al contexto interno del objeto. Por ejemplo, en el caso de `pepita`, el objeto que representa su energía es un colaborador interno.

2. **Como colaborador externo**: son objetos que llegan junto con un mensaje y sólo están disponibles en el contexto de ese mensaje particular. Una vez que el método termina su ejecución, el objeto ya no tiene acceso a ese colaborador externo. En el caso de `pepita`, la cantidad de comida que llega con el mensaje `#comer:` es un colaborador externo.

3. **Como objeto bien conocido**: son objetos que están disponibles de manera global en el sistema. Ejemplos típicos son los números (`1`, `2`, etc.), los booleanos (`true`, `false`), los objetos con nombre propio (como `pepita`), o cualquier otro objeto que el ambiente defina como accesible desde cualquier contexto.

En cualquier caso, el concepto de relación de conocimiento es siempre el mismo: permite a un objeto referirse a otro a través de un nombre y, por lo tanto, poder colaborar con él enviándole mensajes.

¿En qué consiste la estructura de un objeto?

Ahora que ya tenemos una comprensión más clara de algunos conceptos, podemos analizar más profundamente los elementos que necesita un objeto para poder funcionar dentro de nuestro paradigma.

Recordemos algunos de los postulados básicos que hemos establecido:

- Un programa orientado a objetos es un conjunto de objetos que colaboran entre sí.
- Cada objeto representa esencialmente a una cosa de un dominio.
- La esencia de una cosa está vinculada con lo que podemos hacer con ella, su propósito o utilidad.

A partir de ellos, podemos deducir qué características debe tener todo objeto.

En primer lugar, partimos de que un objeto debe expresar, de alguna manera, la esencia de una cosa —y esto define qué podemos hacer con esa cosa—. Pero *lo único que podemos hacer con un objeto es enviarle mensajes*. Entonces, los mensajes que un objeto es capaz de entender deberán corresponderse con la esencia de la cosa que representa, y

determinarán qué puedo hacer con el objeto (es decir, su comportamiento). Por lo tanto, todo objeto debe tener:

1. **Un comportamiento bien determinado**: es lo que sabe hacer el objeto, sus *responsabilidades*, expresado a través de los mensajes que entiende. Debe representar la esencia de la cosa que el objeto está modelando, aquello que la hace ser lo que es.

   El conjunto de mensajes que un objeto entiende forma su *protocolo*. Este protocolo es *público*: otros objetos necesitan conocerlo para poder colaborar con él.

En segundo lugar, cuando un objeto recibe un mensaje que le indica *qué* queremos que haga, necesitamos especificar *cómo* lo va a hacer. Después de todo, el programa tiene que ser ejecutable por la computadora. Esto nos lleva a la necesidad de tener una *implementación* para cada comportamiento que el objeto expone. Dado que lo único que puede hacer un objeto es colaborar con otros, entonces necesitamos definir *qué objetos específicos conoce para poder colaborar* y *qué colaboraciones establece con ellos cuando recibe un mensaje*. Por lo tanto, todo objeto también debe tener:

2. **Una *implementación* para su comportamiento**: es la manera particular en que un objeto decide responder a los mensajes que recibe. Establece *cómo* el objeto hace lo que hace, proporcionando *una posible implementación* para la esencia de la cosa que está representando. Es decir, mientras que el comportamiento (1) define el "qué", la implementación (2) define el "cómo".

   La implementación incluye:
   - Los *métodos* que definen qué hacer para cada mensaje. Vamos a dar una definición precisa de “método” más adelante.
   - Los *colaboradores internos* que el objeto conoce y usa para hacer su trabajo.

   A diferencia del protocolo, la implementación es *privada*: ningún otro objeto puede ni debe conocerla o depender de ella.

Finalmente, para que un objeto pueda colaborar con otro, debe existir una manera de especificar a qué objeto concreto quiere enviarle el mensaje. Es decir, como las colaboraciones son *dirigidas*, entonces tenemos que poder decir "quiero mandarle tal mensaje a *este objeto*" y no a cualquier otro, incluso si son muy similares. Esto implica que cada objeto debe tener una *identidad* única que nos permita distinguirlo de todos los demás (ya habíamos mencionado esto cuando hablamos sobre las relaciones de conocimiento). Por lo tanto, todo objeto también debe tener:

3. **Una identidad**: permite distinguir a ese objeto de cualquier otro. Esto es necesario porque, para poder enviar un mensaje, tengo que definir *a qué objeto* se lo estoy mandando. Luego, cada objeto debe ser identificable, incluso si tiene el mismo comportamiento e implementación que otros objetos; y esta identidad es lo que permite que distintos objetos colaboren entre sí de manera precisa.

El concepto de identidad precede a la programación orientada a objetos, y por lo tanto vale la pena explorarlo un poco más en detalle.

Identidad: ser uno mismo

El concepto de identidad surge a partir de lo que se conoce como el *principio de identidad*[12]:

> **Principio de identidad:** toda cosa es idéntica a sí misma y sólo a sí misma.

La identidad es una propiedad intrínseca e inmutable de cada cosa. No depende de ningún criterio arbitrario ni de las características de la cosa. Dos cosas, por más similares que sean, siempre tendrán identidades diferentes.

Por lo tanto, vemos aquí que en realidad tenemos *dos* razones por las cuales los objetos tienen identidad:

- una razón *técnica*, que ya hemos explicado: la identidad del objeto es lo que permite que los mensajes sean dirigidos y que se puedan establecer relaciones de conocimiento entre los objetos; y
- una razón *conceptual*: como cada objeto representa a una cosa del dominio, y cada cosa tiene una identidad única que la distingue de todas las demás (por el principio de identidad), luego cada objeto en nuestro programa también deberá tener identidad.

Vamos a hacer una aclaración adicional aquí, para no confundir la identidad con otro concepto relacionado pero diferente: *la igualdad*.

Igualdad: compartir características

La *igualdad*, por otro lado, es un concepto relativo que depende de criterios arbitrarios que nosotros definimos. Dos objetos son iguales si, según ciertos criterios de comparación que establezcamos, no nos interesa distinguir a uno del otro.

Por ejemplo, pensemos en dos celulares del mismo modelo: a simple vista, son iguales si tienen el mismo color, la misma pantalla y las mismas especificaciones técnicas. Sin embargo, no son *el mismo* dispositivo (tienen identidades distintas): cada uno tiene su propio número de serie, pueden estar en lugares distintos y ser usados por personas diferentes de forma simultánea.

Podemos considerar también el caso de dos cajas de ahorro en un banco, que en un momento dado tienen exactamente el mismo saldo (digamos $1000). Bajo cierto criterio, podríamos considerarlas iguales. Sin embargo, no son la misma caja de ahorros, y probablemente pertenecen a distintos clientes. Por lo tanto, aunque hoy puedan considerarse iguales según el criterio del saldo, mañana podrían evolucionar de forma independiente: en una se podría depositar dinero mientras que de la otra se podría extraer, dejando de ser "iguales" bajo ese criterio. Sin embargo, en todo momento siguen manteniendo sus *identidades* originales.

[12] https://es.wikipedia.org/wiki/Principio_de_identidad

Entonces, a diferencia de la identidad —que es una propiedad inherente a cada objeto—, la igualdad es una relación entre dos objetos (donde es sólo un caso particular la igualdad de un objeto consigo mismo), y nos permite definir cuándo pueden ser tratados como equivalentes para ciertos propósitos, aunque no sean el mismo objeto.

La igualdad y la identidad en Smalltalk

En Smalltalk, existen dos mensajes para realizar estas comparaciones:

- El mensaje `#==` compara por *identidad*: responde `true` si y sólo si el receptor y el colaborador externo son exactamente el mismo objeto. La implementación de este mensaje está predefinida por el ambiente y no debe ser modificada.
- El mensaje `#=` compara por *igualdad*: su implementación predeterminada está basada en la identidad, pero cada objeto puede redefinirla para establecer sus propios criterios de igualdad[13].

  Por ejemplo:

  ```
  1 = 1.0     "=> true"
  1 == 1.0    "=> false"
  ```

  En este caso, el número entero `1` y el número decimal `1.0` son considerados iguales según su valor numérico, pero definitivamente no son el mismo objeto (tienen identidades diferentes).

📝 Cuando estemos programando, excepto en casos muy puntuales, *vamos a preferir comparar por igualdad* (usando el mensaje `#=`).

❦

En esta sección hemos analizado qué elementos necesita un objeto para funcionar: vimos que todo objeto debe tener un *comportamiento* (expresado a través de los mensajes que entiende), una *implementación* (la forma en que efectivamente responde a esos mensajes) y una *identidad* (que lo distingue de cualquier otro objeto).

Al definir estas características, mencionamos dos conceptos nuevos: el *protocolo*, que representa el conjunto de mensajes que un objeto entiende; y los *métodos*, que definen qué hace un objeto cuando recibe cada mensaje. Ambos conceptos merecen una explicación más detallada.

¿Qué es un protocolo?

Hemos visto que un objeto sabe responder a ciertos mensajes; este conjunto de mensajes forma lo que llamamos el *protocolo* (o interfaz) del objeto.

[13] Si en algún momento redefinimos la igualdad, tenemos que tomar en cuenta que esta relación debe cumplir con las siguientes propiedades:

- Reflexiva: `x = x`.
- Simétrica: si `a = b`, entonces `b = a`.
- Transitiva: si `a = b` y `b = c`, entonces `a = c`.

El **protocolo de un objeto** es el *conjunto de mensajes que ese objeto es capaz de entender y responder*. Representa el comportamiento visible del objeto, la interfaz mediante la cual otros objetos pueden interactuar con él.

Si un objeto recibe un mensaje que no forma parte de su protocolo, de forma predeterminada se producirá un evento conocido como “excepción” (en este caso, una excepción conocida como *MessageNotUnderstood* o "mensaje no comprendido"), que interrumpirá la ejecución del programa.

Al margen: “protocolo” vs. “interfaz”

¿Por qué preferimos la palabra "protocolo" por sobre "interfaz", que es quizás más común de escuchar en programación? Porque "protocolo" tiene una connotación más fuerte: implica un acuerdo que debe ser respetado rigurosamente. Pensemos en los protocolos que existen en otros ámbitos: protocolos médicos, protocolos diplomáticos, protocolos de seguridad. En todos estos casos, el protocolo establece reglas precisas que deben seguirse para que todo funcione correctamente.

De la misma manera, el protocolo de un objeto establece un contrato que debe ser respetado: si quiero interactuar con este objeto, debo hacerlo a través de los mensajes que define su protocolo, no hay otra forma. Si no respeto este protocolo, la comunicación falla y el programa se interrumpe.

El protocolo y la esencia del objeto

El protocolo de un objeto no debe ser arbitrario: refleja y representa la esencia de la cosa que el objeto está modelando. Si vemos un objeto que entiende los mensajes `#saldo`, `#depositar:` y `#extraer:`, podemos reconocer rápidamente que probablemente está representando a una cuenta bancaria.

Esto significa que el protocolo define "qué es" un objeto, y por ello tiende a ser estable a lo largo del tiempo. Mientras que la implementación interna puede cambiar (por ejemplo, podríamos reemplazar completamente cómo el objeto registra las transacciones de depósito y extracción), su protocolo debería permanecer constante. Si un objeto que representa una cuenta bancaria dejara de entender el mensaje `#extraer:`, ¿seguiría siendo una cuenta bancaria? Probablemente no.

📝 Por supuesto, el protocolo de un objeto puede evolucionar en el tiempo, especialmente en las etapas iniciales del desarrollo cuando todavía estamos comprendiendo mejor el dominio. Es decir, una cosa es *la esencia de la cosa del dominio*; otra es *lo que yo entiendo que es esa esencia en un momento de tiempo* (entendimiento que puede ir cambiando a medida que vamos aprendiendo acerca del dominio y confirmando ese entendimiento con los especialistas)*; y otra* es *cómo decidí representarla hasta ese momento*.

Más allá del nombre de los mensajes

Un protocolo va más allá de simplemente definir los selectores de los mensajes que entiende un objeto. También implica una *semántica* asociada a esos mensajes: qué significan y qué efecto esperamos que produzcan cuando se envían.

Por ejemplo, si un objeto representa una golondrina y entiende el mensaje `#volar`, esperamos que cuando le enviemos ese mensaje, su energía disminuya de acuerdo a cierta lógica del dominio.

Esta semántica, lamentablemente, en general no está especificada explícitamente como parte del programa. En muchos casos se documenta a través de comentarios, lo cual puede ser propenso a errores y costoso en términos de mantenimiento. Algunos lenguajes proveen mecanismos más sofisticados para especificar formalmente esta semántica, por ejemplo en términos de aserciones conocidas como *precondiciones* y *postcondiciones*. Aunque esto último no está soportado por Cuis-Smalltalk, nosotros vamos a preferir especificar esta semántica por medio de pruebas automatizadas en lugar de usar comentarios.

Las pruebas van a jugar un papel fundamental en la especificación del protocolo de nuestros objetos, ya que proporcionan una forma ejecutable de documentar y luego verificar que un objeto se comporta como esperamos cuando recibe ciertos mensajes.

Protocolo público vs. implementación privada

Como ya hemos comentado, el protocolo de un objeto es *público*, mientras que su implementación es *privada*. Esto significa que:

1. Cualquier otro objeto puede conocer y utilizar el protocolo (es decir, enviar los mensajes que el objeto entiende).
2. Ningún objeto externo debería conocer ni depender de cómo están implementados esos mensajes.

Esta separación entre el "qué" (el protocolo) y el "cómo" (la implementación) es una de las ideas fundamentales del paradigma de objetos, y promueve lo que llamamos *encapsulamiento*: la capacidad de ocultar los detalles internos de un objeto, exponiendo sólo su comportamiento esencial.

¿Qué es un método?

Ya vimos que los objetos responden a mensajes, y el conjunto de mensajes que un objeto entiende forma su protocolo. Pero, ¿qué sucede concretamente cuando un objeto recibe un mensaje? ¿Cómo sabe qué hacer?

> Un **método** es *la secuencia de colaboraciones que un objeto establece con otros objetos para responder a un mensaje determinado*. En otras palabras, es la implementación concreta del comportamiento asociado a un mensaje.

Recordemos nuestro principio fundamental: en un programa orientado a objetos, lo único que puede hacer un objeto es enviar mensajes a otros objetos. Por lo tanto, cuando un objeto recibe un mensaje, lo que va a hacer es, a su vez, enviar mensajes a otros objetos (sus colaboradores) para resolver la tarea solicitada.

Por ejemplo, si tenemos a `pepita` la golondrina, y le enviamos el mensaje `#volar`, el método asociado a este mensaje podría involucrar enviar el mensaje `#-` a su colaborador interno `energía`, pasándole el esfuerzo del vuelo.

La estructura de un método

En su forma más básica, un método consiste en una serie de colaboraciones (envíos de mensajes) con otros objetos. Estas colaboraciones se ejecutan secuencialmente y, eventualmente, el método puede terminar retornando un objeto como respuesta al mensaje original.

El siguiente es un ejemplo de método en Smalltalk:

```
volar
    energía ← energía - (10 * caloría)
```

Este método para el mensaje `volar` consiste en una colaboración que depende del resultado de otra: le envía el mensaje `* caloría` a `10`, luego el mensaje `- (10 * caloría)` al objeto que conoce a través del nombre `energía`, y por último nombra la respuesta final como `energía` (es decir, actualiza quién es su colaborador interno con ese nombre).

Cuando un objeto ejecuta un método en respuesta a un mensaje, tiene acceso a todos los objetos que conoce para poder colaborar: sus colaboradores internos (como `energía` en el caso de `pepita`), los colaboradores externos (que llegan junto con el mensaje y que están disponibles únicamente durante la ejecución del método), el propio objeto (a través del nombre especial `self`), y los objetos bien conocidos (como los números).

La privacidad de la implementación

Así como el protocolo de un objeto es público, su implementación es privada. Esto significa que solo el propio objeto puede ejecutar sus métodos; ningún objeto externo puede "forzar" la ejecución de un método de otro objeto. La única forma de que un método se ejecute es que el objeto decida hacerlo al recibir el mensaje correspondiente.

Esta característica es fundamental para el encapsulamiento: cada objeto decide por sí mismo cómo implementar su comportamiento, y esta implementación puede cambiar sin afectar a los demás objetos, siempre que se mantenga el protocolo.

Resumen

A lo largo de este documento, hemos ido construyendo una definición rigurosa del paradigma de objetos a partir de sus postulados elementales. Vamos a resumir los conceptos principales:

Programa orientado a objetos

Un **programa orientado a objetos** es un conjunto de objetos que colaboran entre sí enviándose mensajes. En este paradigma, todo es un objeto y lo único que pueden hacer los objetos es enviar y recibir mensajes. Esta definición es pura y minimal: no existen "datos", "funciones", "estructuras de control" ni otros elementos computacionales que no sean objetos.

Objeto

Un **objeto** es una representación esencial de una cosa de un dominio de interés. Es el elemento computacional básico con el que construimos nuestros programas.

Para entender a qué nos referimos con esto, tuvimos que definir:

- **Cosa:** todo aquello acerca de lo cual podemos decir algo. Esta definición lingüística nos permite considerar como "cosas" tanto elementos concretos (una mesa, una computadora) como abstractos (un número, un préstamo, un error).
- **Dominio**: conjunto de cosas. El dominio de interés es el conjunto específico de cosas relevantes para el problema que estamos resolviendo. Como programadores, típicamente trabajamos con dominios creados por el ser humano para propósitos específicos (bancos, hospitales, librerías, etc.).
- **Esencia** (de una cosa): es lo que hace que esa cosa sea lo que es. Desde una perspectiva pragmática, la esencia de una cosa radica en para qué sirve, qué podemos hacer con ella.
- **Representación esencial**: modelo no arbitrario, que captura la esencia de una cosa. Revela lo que esa cosa es y para qué sirve.

Luego, deducimos que todo objeto tiene:

- Un **comportamiento** bien determinado (qué sabe hacer, expresado a través de los mensajes que entiende).
- Una **implementación** para ese comportamiento (cómo lo hace, mediante métodos y colaboradores internos).
- Una **identidad** única que lo distingue de cualquier otro objeto.

Identidad vs. igualdad

La **identidad** es una propiedad inmutable e inherente a cada objeto, que lo diferencia del resto. Sigue el principio: "toda cosa es idéntica a sí misma y sólo a sí misma".

La **igualdad**, en cambio, es un concepto relativo que depende de criterios arbitrarios que nosotros definimos. Dos objetos son iguales si, según ciertos criterios de comparación, no nos interesa distinguir a uno del otro.

Colaboración

Una **colaboración** es la interacción concreta de un objeto con otro mediante el envío de un mensaje. Para que un objeto pueda colaborar con otro, debe conocerlo.

Toda colaboración tiene un emisor, un receptor, un mensaje y una respuesta.

Colaborador

Un **colaborador** es un objeto conocido que cumple un determinado rol.

Puede ser:

- **Interno**: siempre disponible para el objeto que lo conoce.
- **Externo**: llega junto con un mensaje y solo está disponible en el contexto de ese mensaje.
- **Bien conocido**: disponible globalmente en el sistema.

Mensaje

Un **mensaje** es la manera en que un objeto le solicita a otro que haga algo. Especifica qué queremos que el objeto haga, pero no cómo debe hacerlo.

Los mensajes se componen de:

- Un **selector**, que nombra al mensaje (como `#size`, `#+` o `#between:and:`)
- Cero, uno o más **colaboradores externos**: objetos que se incluyen como parte del mensaje para que el receptor pueda utilizarlos.

Protocolo e implementación

Un **protocolo** es el conjunto de mensajes que un objeto es capaz de entender y responder. Representa el comportamiento visible del objeto y establece los contratos que deben ser respetados.

El protocolo es público y refleja la esencia de la cosa que el objeto está representando. Define "qué es" un objeto, y tiende a ser estable a lo largo del tiempo.

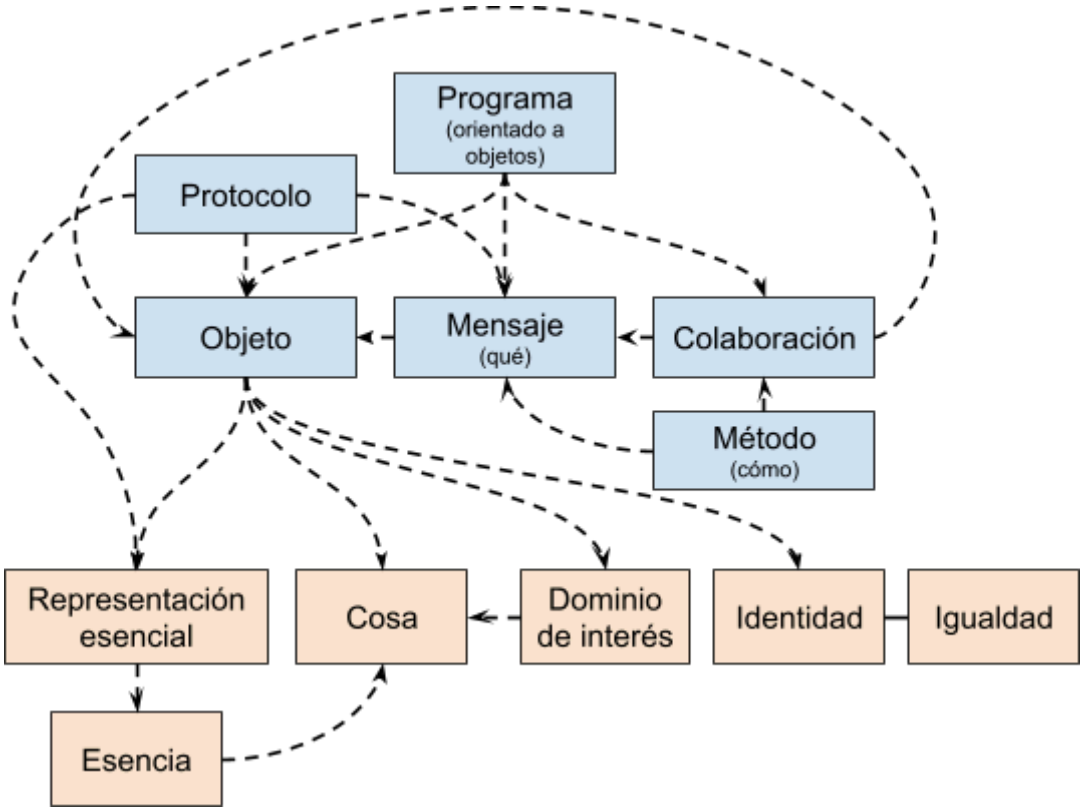
A diferencia del protocolo, la **implementación** del objeto (sus colaboradores internos y sus métodos) es privada. Esto permite que cada objeto pueda cambiar cómo implementa su comportamiento sin afectar a los demás, siempre que mantenga el mismo protocolo.

Método

Un **método** es la secuencia de colaboraciones que un objeto establece con otros objetos para responder a un mensaje determinado. Es la implementación concreta del comportamiento asociado a un mensaje.

Esquema conceptual

Muestra los conceptos principales, y de cuáles otros depende su definición:



El paradigma como forma de pensar

A modo de cierre, es importante entender que nuestro objetivo no es aprender tecnología. Es decir, el foco no está puesto en un determinado lenguaje y/o ambiente de programación, ya que hacer esto puede resultar contraproducente. Una razón es que, a lo largo de nuestro desarrollo profesional como programadores, vamos a necesitar utilizar diversas herramientas tecnológicas con diferencias importantes entre ellas. Además, las tecnologías varían de manera continua, es decir, cambian constantemente a través del tiempo. Es importante también que pensemos en que toda tecnología nos puede ofrecer ventajas, pero también nos impone limitaciones. No existe la tecnología perfecta. Por otro lado, nuestro objetivo es aprender a construir buenos programas. En ese sentido, no podemos culpar a las herramientas que hemos usado si nuestros programas están mal construidos. Los buenos programas son construidos por buenos programadores.

Entonces, si queremos empezar a pensar conceptualmente —en lugar de enfocarnos en una tecnología específica—, puede ser útil considerar qué consecuencias tendría en nuestro oficio la posibilidad de despreocuparnos de las limitaciones que nos impone la tecnología que usamos. Esto sería posible sólo en una situación en la que estemos usando tecnología perfecta, pero ya hemos establecido que esto es imposible. Sabemos que no es algo que podamos alcanzar, *pero querríamos hacerlo*. En otras palabras, estamos planteando ejercer nuestro oficio asumiendo una perfección utópica.

En este intento de acercarnos a esa utopía, vamos a construir un enfoque conceptual, también llamado *paradigma*, que nos permita construir buenos programas de manera independiente de la tecnología que nos toque —o que decidamos— utilizar. Este marco conceptual también nos va a permitir evaluar la tecnología disponible e identificar sus limitaciones para poder superarlas. Eventualmente, si lo disponible no resultara suficiente para hacer nuestro trabajo como mejor nos pareciera, podremos mejorarlo y/o ampliarlo programando mejores herramientas. Si podemos construir programas para otros, ¿por qué no para nosotros mismos?

En nuestro caso, el enfoque conceptual incluye fuertemente al paradigma orientado a objetos, aunque es más abarcativo. Ciertamente, vamos a utilizar técnicas —*Test-Driven Development* (TDD)— y herramientas —Smalltalk y Cuis-University— para poner en práctica dicho enfoque o marco conceptual. Pero no alcanza con aprender una determinada gramática o el uso de ciertas herramientas. Bajo el paraguas de la Orientación a Objetos, encontramos muchos lenguajes que tendremos que saber utilizar de la mejor manera posible —Java, Python, JavaScript, C#, Swift, entre otros—. Si elegimos Smalltalk como lenguaje y ambiente de programación, es porque es el que mejor acompaña a la Orientación a Objetos. Esto nos permite delinear una manera de “pensar con objetos”, de observar los dominios sobre los que vamos a programar “con ojos de objetos”. De este modo, podremos identificar de manera natural qué objetos y qué colaboraciones entre ellos representan de manera adecuada nuestro dominio de interés, independientemente del lenguaje que nos toque utilizar.

Es por todo esto que podemos intuir el concepto de paradigma de programación como una manera consensuada por una comunidad de programadores de “observar” las situaciones que se dan en nuestros dominios de interés, pensar acerca de ellas y expresar nuestro aprendizaje programando. De eso se trata nuestro oficio, de un aprendizaje continuo.

Referencias

Frege, G. (2017). *Escritos lógico-filosóficos* (A. Gómez-Lobo, Ed.). Colihue.

Quine, W. V. O. (1980). *From a logical point of view : 9 logico-philosophical essays*. Harvard University Press.